

03 考试题型闭卷训练与验收

这份笔记不重复讲理论，而是把字符串、数据结构、函数、zip、推导式、lambda 转成考试能力。

考试做题框架

看到数据结构题，先写 7 个判断：

1. 整体是什么类型？
2. 每个元素是什么类型？
3. 每轮循环拿到什么？
4. 需要累计什么变量？
5. 函数接收什么？
6. 函数返回什么？
7. 是否需要排序、筛选或转换？

一、程序输出题

先遮住答案，自己写预测结果。

题 1：append 与 extend

```
data = [1, 2]
data.append([3, 4])
print(data)
```

题 2：sort 返回值

```
nums = [3, 1, 2]
result = nums.sort()
print(nums)
print(result)
```

题 3：列表引用

```
a = [1, 2]
b = a
```

```
b.append(3)
print(a)
```

题 4：单元素元组

```
a = (1)
b = (1,)
print(type(a))
print(type(b))
```

题 5：字典默认遍历键

```
student = {"name": "Tom", "score": 80}
for item in student:
    print(item)
```

题 6：d.get

```
counts = {}
ch = "a"
counts[ch] = counts.get(ch, 0) + 1
print(counts)
```

题 7：集合去重

```
nums = [1, 2, 1, 3]
print(set(nums))
```

题 8：zip 最短序列

```
print(list(zip(["A", "B"], [1, 2, 3])))
```

题 9：enumerate(start=1)

```
names = ["Tom", "Jerry"]
for i, name in enumerate(names, start=1):
    print(i, name)
```

题 10：推导式判断条件

```
n = 5
result = [i for i in range(1, n + 1) if n % 2 == 0]
print(result)
```

题 11: lambda + max

```
students = [
    {"name": "Tom", "score": 80},
    {"name": "Jerry", "score": 92},
]
top = max(students, key=lambda student: student["score"])
print(top)
```

题 12: *args 与 **kwargs

```
def show(*args, **kwargs):
    print(args)
    print(kwargs)

show(1, 2, name="Tom")
```

题 13: 默认列表参数

```
def add(name, students=[]):
    students.append(name)
    return students

print(add("Tom"))
print(add("Jerry"))
```

题 14: 递归输出顺序

```
def f(n):
    if n == 0:
        return
    print(n)
    f(n - 1)

f(3)
```

程序输出题答案区

答 1

```
[1, 2, [3, 4]]
```

易错原因：`append` 把整个列表作为一个元素。

答 2

```
[1, 2, 3]  
None
```

易错原因：`sort()` 原地排序，返回 `None`。

答 3

```
[1, 2, 3]
```

易错原因：`b = a` 是同一个列表。

答 4

```
<class 'int'>  
<class 'tuple'>
```

易错原因：单元素元组靠逗号。

答 5

```
name  
score
```

易错原因：字典默认遍历键。

答 6

```
{ 'a': 1 }
```

答 7

```
{1, 2, 3}
```

集合输出顺序不应依赖。

答 8

```
[('A', 1), ('B', 2)]
```

答 9

```
1 Tom  
2 Jerry
```

答 10

```
[]
```

易错原因：判断的是 `n % 2 == 0`，不是 `i`。

答 11

```
{'name': 'Jerry', 'score': 92}
```

`max` 返回原始学生字典。

答 12

```
(1, 2)  
{'name': 'Tom'}
```

答 13

```
['Tom']  
['Tom', 'Jerry']
```

易错原因：默认列表只创建一次。

答 14

3
2
1

二、填空题

题 1

把 "80 90 70" 拆成列表: `"80 90 70"._____()`

题 2

把 ["a", "b", "c"] 拼成 "a,b,c": `", "._____ (items)`

题 3

遍历字典键值对: `for key, value in d._____():`

题 4

遍历列表时同时拿编号: `for i, item in _____(items, start=1):`

题 5

把两个列表配对: `for name, score in _____(names, scores):`

题 6

两个列表转字典: `dict(_____(names, scores))`

题 7

筛选偶数列表推导式: `[x for x in nums if _____]`

题 8

按成绩排序: `sorted(students, key=lambda student: student["_____"])`

题 9

调用时拆开列表: `add(____numbers)`

题 10

函数返回结果关键字： `_____ result`

答案： `split`、 `join`、 `items`、 `enumerate`、 `zip`、 `zip`、 `x % 2 == 0`、 `score`、 `*`、 `return`。

三、改错题

每题按“错误现象 → 错误原因 → 最小修改”来做。

题 1：split 后忘记转整数

```
scores = "80 90 70".split()
print(sum(scores))
```

正确：

```
scores = []
for part in "80 90 70".split():
    scores.append(int(part))
print(sum(scores))
```

题 2：join 连接整数列表

```
nums = [1, 2, 3]
print(",".join(nums))
```

正确：

```
nums = [1, 2, 3]
parts = []
for num in nums:
    parts.append(str(num))
print(",".join(parts))
```

题 3：字典直接与整数相加

```
student = {"name": "Tom", "score": 80}
print(student + 10)
```

正确：

```
print(student["score"] + 10)
```

题 4：推导式判断错变量

```
n = 10
result = [i for i in range(1, n + 1) if n % 2 == 0]
```

正确：

```
result = [i for i in range(1, n + 1) if i % 2 == 0]
```

题 5：循环时删除原列表

```
nums = [1, 2, 3, 4]
for num in nums:
    if num % 2 == 0:
        nums.remove(num)
```

正确：

```
result = []
for num in nums:
    if num % 2 != 0:
        result.append(num)
```

题 6：忘记 return

```
def add(a, b):
    result = a + b

print(add(1, 2))
```

正确：

```
def add(a, b):
    return a + b
```

题 7：默认参数使用空列表


```
def add_student(name, students=[]):  
    students.append(name)  
    return students
```

正确：

```
def add_student(name, students=None):  
    if students is None:  
        students = []  
    students.append(name)  
    return students
```

题 8：平均值在循环中计算

```
total = 0  
for score in scores:  
    total += score  
average = total / len(scores)
```

正确：

```
total = 0  
for score in scores:  
    total += score  
average = total / len(scores)
```

题 9：使用累计总分比较最高分

```
highest = scores[0]  
total = 0  
for score in scores:  
    total += score  
    if total > highest:  
        highest = total
```

正确：

```
highest = scores[0]  
for score in scores:  
    if score > highest:  
        highest = score
```

题 10: **student 键与参数名不一致

```
def show_student(name, score):  
    print(name, score)  
  
student = {"username": "Tom", "score": 80}  
show_student(**student)
```

正确：

```
student = {"name": "Tom", "score": 80}  
show_student(**student)
```

四、简单编程题

题 1：字符串转成绩列表

输入：

```
80 90 75 60
```

要求输出整数列表。

```
text = "80 90 75 60"  
scores = []  
for part in text.split():  
    scores.append(int(part))  
print(scores)
```

题 2：成绩去重并降序

```
scores = [80, 90, 80, 70]  
unique_scores = list(set(scores))  
unique_scores.sort(reverse=True)  
print(unique_scores)
```

说明：使用 `set` 不保留原顺序，只适合不要求原顺序的去重。

题 3：字符频率统计

```
text = "abc1ab"
counts = {}
for ch in text:
    counts[ch] = counts.get(ch, 0) + 1
print(counts)
```

题 4：两个列表生成字典

普通循环：

```
names = ["Tom", "Jerry"]
scores = [80, 92]
result = {}
for name, score in zip(names, scores):
    result[name] = score
```

dict(zip())：

```
result = dict(zip(names, scores))
```

字典推导式：

```
result = {name: score for name, score in zip(names, scores)}
```

题 5：筛选及格学生

```
students = [
    {"name": "Tom", "score": 80},
    {"name": "Alice", "score": 55},
]

passed = []
for student in students:
    if student["score"] >= 60:
        passed.append(student)
print(passed)
```

题 6：按成绩排序

元组列表：

```
students = [("Tom", 80), ("Jerry", 92)]
print(sorted(students, key=lambda item: item[1], reverse=True))
```

字典列表：

```
students = [{"name": "Tom", "score": 80}, {"name": "Jerry", "score": 92}]
print(sorted(students, key=lambda student: student["score"], reverse=True))
```

zip 结果：

```
names = ["Tom", "Jerry"]
scores = [80, 92]
print(sorted(zip(names, scores), key=lambda item: item[1], reverse=True))
```

五、最终综合题：学生成绩管理

输入要求

输入学生人数，然后输入：

姓名,成绩

例如：

```
Tom,80
Jerry,92
Alice,55
```

功能要求

1. 使用 `strip()` 清理输入。
2. 使用 `split(",")` 拆分。
3. 将成绩转为整数。
4. 每个学生保存为字典。
5. 所有学生保存到列表。
6. 输出最高分学生。
7. 输出最低分学生。
8. 输出平均分。

9. 输出及格人数。
10. 返回不及格学生列表。
11. 按成绩降序排列。
12. 成绩相同时按姓名排序。
13. 使用 `show_student(name, score)`。
14. 至少一次通过 `**student` 调用。
15. 使用 `join()` 生成排名文本。
16. 不使用全局变量。

A. 基础循环版

```
def parse_student(line):
    name, score_text = line.strip().split(",")
    return {"name": name, "score": int(score_text)}

def input_students():
    count = int(input("请输入学生人数: "))
    students = []

    for i in range(count):
        line = input("请输入姓名,成绩: ")
        student = parse_student(line)
        students.append(student)

    return students

def analyze_students(students):
    top = students[0]
    low = students[0]
    total = 0
    passed = 0

    for student in students:
        score = student["score"]
        total += score

        if score > top["score"]:
            top = student

        if score < low["score"]:
            low = student
```

```

        if score >= 60:
            passed += 1

average = total / len(students)

return {
    "top": top,
    "low": low,
    "average": average,
    "passed": passed,
}

def get_failed_students(students):
    failed = []
    for student in students:
        if student["score"] < 60:
            failed.append(student)
    return failed

def sort_students(students):
    return sorted(students, key=lambda student: (-student["score"],
student["name"]))

def show_student(name, score):
    return f"{name}:{score}"

def format_students(students):
    lines = []
    for student in students:
        lines.append(show_student(**student))
    return "\n".join(lines)

def main():
    students = input_students()
    result = analyze_students(students)
    failed = get_failed_students(students)
    sorted_students = sort_students(students)

    print("最高分: ", show_student(**result["top"]))
    print("最低分: ", show_student(**result["low"]))
    print("平均分: ", result["average"])

```

```
print("及格人数: ", result["passed"])
print("不及格学生: ", failed)
print("排名: ")
print(format_students(sorted_students))

if __name__ == "__main__":
    main()
```

B. 简化提升版

考试中不会简化写法时，优先写基础循环版。

```
def parse_student(line):
    name, score_text = line.strip().split(",")
    return {"name": name, "score": int(score_text)}

def analyze_students(students):
    return {
        "top": max(students, key=lambda student: student["score"]),
        "low": min(students, key=lambda student: student["score"]),
        "average": sum(student["score"] for student in students) /
len(students),
        "passed": sum(1 for student in students if student["score"] >= 60),
    }

def get_failed_students(students):
    return [student for student in students if student["score"] < 60]

def sort_students(students):
    return sorted(students, key=lambda student: (-student["score"],
student["name"]))

def show_student(name, score):
    return f"{name}:{score}"

def format_students(students):
    return "\n".join(show_student(**student) for student in students)
```

六、100 分验收

模块	分值	通过要求
基础概念	15	能说清四种结构区别
方法与转换	15	会 <code>split/join</code> 、结构互转
程序输出	15	正确率 80% 以上
改错	15	能指出最小修改
函数与数据结构	15	会参数、返回值、 <code>*args/**kwargs</code>
综合编程	25	能写学生成绩综合题主体

通过标准：

85 分以上：该部分通过

70~84 分：补错题后再次验收

70 分以下：重新练遍历、转换、参数和 `return`

验收必须覆盖：

四种结构区别

常见增删改查

`split/join`

结构互转

`zip/enumerate`

三种推导式

`*args/**kwargs`

`lambda + key`

可变参数陷阱

学生综合题

七、一天 7~8 小时执行时间表

阶段	建议时间	学习内容	闭卷产出	通过标准
第 1 阶段	40 分钟	字符串输入处理	写出 <code>strip/split/int/join</code> 例题	能处理 <code>"Tom,80"</code>

阶段	建议时间	学习内容	闭卷产出	通过标准
第 2 阶段	60 分钟	四种数据结构	写结构对比和增删改查	能选对结构
第 3 阶段	40 分钟	结构互转	写 <code>dict(zip())</code> 和字典计数	不混淆键和值
第 4 阶段	60 分钟	<code>zip/enumerate/推导式</code>	写 6 个小题	能解释每轮变量
第 5 阶段	60 分钟	函数参数与返回值	写 5 个函数	不把 <code>print</code> 当 <code>return</code>
第 6 阶段	45 分钟	<code>lambda/排序/统计</code>	写按成绩排序	能解释 <code>key</code>
第 7 阶段	60 分钟	阅读与改错	做输出题和改错题	正确率 80%
第 8 阶段	90 分钟	综合题	写学生成绩管理主体	函数拆分完整
第 9 阶段	30 分钟	验收与复盘	登记错题	明确是否通过

[返回字符串与数据结构基础](#)

[继续学习函数综合](#)